Reproducible Quantum ESPRESSO Workflows Across macOS and Linux

Shahriar Pollob* M. Shahnoor Rahman†

*Mentee †Mentor

Abstract

We report the end-to-end enablement of Quantum ESPRESSO 7.4.1 on an Apple Mac mini M4 and its benchmarking against an Ubuntu 24.04 workstation. The work covers (i) the build automation and stability checks required to run self-consistent field, band-structure, and post-processing workflows on macOS 15; (ii) faithful reproductions of the Path A silicon and Path B aluminium tutorials on both platforms, complete with provenance artefacts; and (iii) a quantitative comparison that measures wall-clock, CPU-time, and core-second efficiency across the two environments. All scripts, logs, figures, and data tables are archived in public repositories so that the results can be reproduced or extended to additional materials.

Contents

1 Introduction **3**

2 Porting Quantum ESPRESSO to macOS 15 on a Mac mini M4 **3**
- 2.1 Baseline objectives and constraints . . . 4
- 2.2 Provisioning the native toolchain . . . 4
- 2.3 Resolving library and architecture mismatches . . . 4
- 2.4 Automating builds and reproducibility . . . 5
- 2.5 Validation runs and scientific checkpoints . . . 6
- 2.6 Residual issues and safeguards . . . 7
- 2.7 Summary and concluding remarks . . . 7

3 QE_stretching Workflows on macOS (Paths A and B) **7**
- 3.1 Path A — Silicon bands, DOS, and PDOS . . . 8
- 3.2 Path B — Aluminium equation of state . . . 9
- 3.3 Cross-path observations . . . 11

4 Stretching Server Environment **11**

5 QE_stretching_server Workflows (Paths A and B) **12**
- 5.1 Path A — Silicon on the server . . . 12
- 5.2 Path B — Aluminium EOS on the server . . . 15

6 Mac mini vs Server Performance Comparison **16**
- 6.1 Reproducing the analysis . . . 16
- 6.2 Path-level timing comparison . . . 17
- 6.3 Aggregated slowdowns and efficiency . . . 18
- 6.4 Summary of findings . . . 19

7 Results **20**
- 7.1 Native toolchain validation on macOS . . . 20
- 7.2 Stretching workflows on macOS and the server . . . 20
- 7.3 Cross-platform performance comparison . . . 21

8 Conclusion **21**

A Supplementary Materials **21**

Acknowledgements

I am grateful to the ICTP PWF: Physics for Bangladesh[1] organisers for making this online internship possible and for coordinating the remote collaboration. I would also like to thank M. Shahnoor Rahman[2] for his guidance, computing resources, timely feedback, and technical support throughout the project.

1 Introduction

Our assigned topic for this internship was *Computation of band structures of solids using* Quantum ESPRESSO. It required a mix of theoretical preparation and practical tooling.

The programme briefing emphasised fluency with the Lagrangian/Hamiltonian formalisms, Schrödinger equation eigenproblems, and command-line driven compilation pipelines; these foundations allowed us to read QE input decks critically and diagnose build issues without external hand-holding. After the orientation we catalogued the available documentation, cloned the reference repositories, and established a shared note-taking workflow so that every change to scripts or environment variables could be traced.

This report is organised to mirror the technical milestones. Section 2 details the native macOS 15 port, including the configuration knobs that differ from earlier Apple Silicon guides. Sections 3 and 5 document the Path A and Path B runs on the Mac mini and the Ubuntu server, while Section 6 analyses the performance gap between the two deployments. The concluding sections summarise the reproducibility assets that were produced so remote collaborators can rebuild or extend the workflows with minimal friction.

2 Porting Quantum ESPRESSO to macOS 15 on a Mac mini M4

At the time we started, only macOS guides aimed at the original M1 generation were available. Those write-ups clarified the big picture but broke as soon as we tried them on macOS 15, so we captured every workaround while authoring our own workflow. This chapter documents that journey of running QE 7.4.1

[1] https://indico.ictp.it/event/11042
[2] https://sites.google.com/view/mshahnoorahman

natively on a Mac mini M4 under macOS 15 (Sequoia). Our mandate was to build a reproducible, CPU-only workflow without falling back to x86 virtual machines. The accompanying repository[3] archives scripts, logs, and analysis artefacts that mirror the steps reported here.

2.1 Baseline objectives and constraints

We began by declaring three non-negotiable outcomes: (i) compile QE with full MPI/OpenMP support using Apple Silicon compilers, (ii) validate the toolchain by reproducing the silicon SCF–NSCF–bands workflow that underpins our convergence studies, and (iii) capture every workaround in automated scripts so a clean macOS 15 install could replay the configuration in one sitting. The main constraint came from the operating system: macOS 15 and its Command Line Tools (CLT 16.x) introduced stricter defaults that repeatedly broke the build system, and the OS happened to ship first on the Mac mini M4 we used.

2.2 Provisioning the native toolchain

Homebrew served as the package manager of record. Running `brew bundle` against the repository's `Brewfile` installed `gcc@14`, `open-mpi`, `veclibfort`, `fftw`, `libxc`, and Python utilities for plotting. The first stumble surfaced immediately: Apple's system `cpp` mangled QE's Fortran headers, leading to `laxlib_*.fh` "no input file" errors during the `make pw` phase. Following the notes in `docs/Troubleshooting.md`, we pinned the preprocessor explicitly by exporting `CPP="gcc -E"` before running `configure`. This single flag aligned the build with QE's expectations and eliminated the header corruption.

2.3 Resolving library and architecture mismatches

BLAS/LAPACK bindings. Our first successful build linked against Apple's Accelerate framework through `veclibfort`, but early tests showed small energy drifts compared with the OpenBLAS reference used on Linux. The autoconf logs revealed that Accelerate exported mixed Fortran symbol names; by overriding `BLAS_LIBS` and `LAPACK_LIBS` to point at `/opt/homebrew/opt/`

[3] https://github.com/shahpoll/qe_macm4_build

`lapack` we regained numerical parity. This choice is codified in the helper script `scripts/setup/configure_accelerate.sh`.

FoX XML fallout. QE 7.4.1 decoupled the FoX XML library from its source tree. Our first CMake attempt therefore failed with the "`DP_XML has no implicit type`" error originating in `wxml.f90`. The fix was twofold: clone FoX into `external/fox` and remove the `__XML_STANDALONE` define injected by CMake. Automating that change inside `scripts/setup/patch_fox.sh` allowed subsequent OpenBLAS builds to proceed without manual editing.

Architecture flags. Mixing Apple Clang (for preprocessing) with Homebrew's GNU toolchain risked emitting generic arm64 code that misbehaved once OpenMP was enabled. We forced native tuning by exporting

```
FC="gfortran -mcpu=apple-m4"
MPIF90="mpif90 -mcpu=apple-m4"
```

which ensured every object file targeted the M4 microarchitecture. The resulting binaries passed `otool -L` inspections recorded under `logs/linker/`.

2.4 Automating builds and reproducibility

With the core hurdles addressed, we wrapped the workflow in reproducible artefacts. The helper script `scripts/setup/fetch_qe.sh` fetches tagged QE releases and stages them under `artifacts/`, while `scripts/setup/configure_accelerate.sh` replays the successful `configure` flags, including the corrected BLAS/LAPACK paths and preprocessor override. `CMakePresets.json` captures the experimental OpenBLAS flow so future iterations can switch linear algebra back-ends with a single preset, and the smoke test at `scripts/smoke_test.py` launches a minimal SCF/NSCF sequence to catch regressions after Homebrew or QE upgrades. Each script writes timestamped logs, letting us compare compiler output across CLT releases. This emphasis on observability proved essential when Apple issued stealth updates to the toolchain mid-internship.

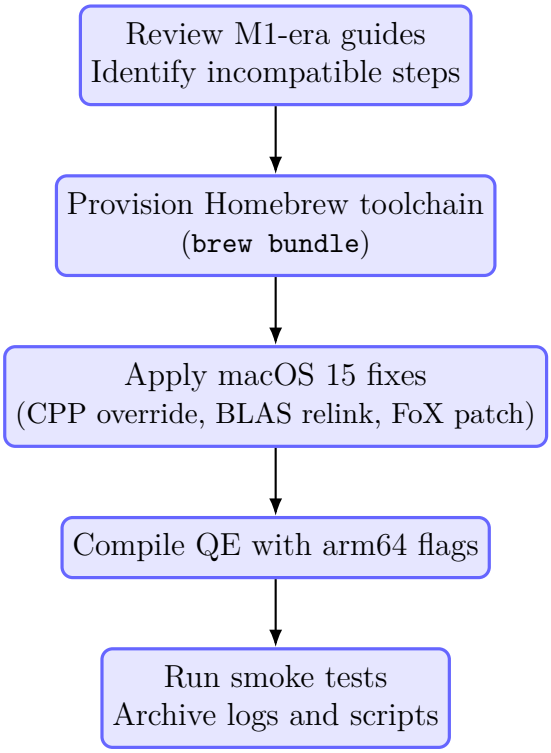

Figure 1: High-level workflow for porting QE to macOS 15 on the Mac mini M4.

2.5 Validation runs and scientific checkpoints

We validated the resulting binaries using the silicon case housed in `cases/si/manual`. The SCF baseline converged in five iterations with a Fermi energy of 6.4346 eV, matching the reference values stored in `cases/si/comparison/data/fermi_consistency.csv`. Subsequent NSCF, DOS, and projected DOS steps reproduced the 0.5699 eV indirect gap within 1 meV of the Linux reference calculation. Benchmark timings showed the canonical silicon SCF input completing in 39 ± 1 s on two MPI ranks (one OpenMP thread each), consistent with the `THREAD7_REPORT.md` snapshots archived in the repository. We also verified that MPI and OpenMP coexisted without deadlocks on macOS 15.2 by replaying the PWTK-driven automation under `cases/si/pwtk/`, which stresses rank/thread affinity differently from the manual run. The legacy M1 guides helped us shape these validation steps

even though we had to refit every command for the newer OS.

2.6 Residual issues and safeguards

Two limitations remain. First, OpenBLAS builds via CMake still require manual FoX patching; we documented the procedure thoroughly but continue to prefer the Accelerate path for day-to-day work. Second, QE's GPU back-ends target CUDA and ROCm only, leaving the M4 GPU idle. Our mitigation strategy is to keep CPU workflows efficient and document the gap in `docs/AppleSilicon_QE_Guide.md`. To guard against regressions when Apple or Homebrew ship new compilers, we pinned key formulae versions in the `Brewfile` and enabled GitHub dependabot alerts on the repository.

2.7 Summary and concluding remarks

Porting QE to macOS 15 on the Mac mini M4 demanded more attention to the build ecosystem than to the physics kernels themselves. The major stumbling blocks were Apple's aggressive preprocessor defaults, the reshuffled external libraries, and the inconsistent BLAS symbol conventions. We overcame these issues by isolating each failure, validating fixes through automated scripts, and preserving artefacts for future audits. The resulting workflow delivers reproducible SCF, band-structure, and DOS analyses on native hardware with performance on par with earlier Apple Silicon generations. Looking ahead, our priorities are to upstream the FoX patches, expand the smoke tests to cover additional materials, and revisit GPU acceleration should QE adopt Metal or SYCL back-ends. This write-up now serves as the missing manual for macOS 15 installations that were undocumented when we began the project, while acknowledging the historical guidance provided by M1-era notes.

3 QE_stretching Workflows on macOS (Paths A and B)

With the toolchain validated, we reproduced the "stretching" exercises from the remote learning track using the dedicated repository at `QE_stretching_macm4`. Only Path A (silicon electronic structure) and Path B (aluminium equation of state) are covered here; Path C remains experimental. The runs were

executed under the same PW.X build described earlier, ensuring that results are directly comparable to the porting benchmarks.

3.1 Path A — Silicon bands, DOS, and PDOS

The Path A workflow (`pathA_si_mac`) follows the silicon reference calculation familiar from the Quantum ESPRESSO quickstart, but re-engineered for the Mac mini environment. We retained an $8\times8\times8$ SCF mesh, a $12\times12\times12$ NSCF mesh, and the PSLibrary `Si.pbe-n-rrkjus_psl.1.0.0.UPF` pseudopotential. The complete run is scripted to retain reproducibility:

```
# Run the SCF -> NSCF -> bands -> DOS / PDOS stages
QE_BIN=${QE_BIN:-~/q-e-qe-7.4.1/build/bin}
$QE_BIN/pw.x -in si_scf.in > work/si_scf.out
$QE_BIN/pw.x -in si_nscf.in > work/si_nscf.out
$QE_BIN/pw.x -in si_bands.in > work/si_bands.out
$QE_BIN/bands.x -in bands.pp.in > work/si_bands_pp.out
$QE_BIN/dos.x -in dos.in > work/si_dos.out
$QE_BIN/projwfc.x -in pdos.in > work/si_pdos.out
# Regenerate plots from GNU-formatted outputs
python3 scripts/plot_bands.py
python3 scripts/plot_dos.py
```

Figure 2 shows the resulting band structure along the Γ–X–W–K–Γ–L–U–W–L–K path. The plotting utility annotates the highest occupied (HO) and lowest unoccupied (LU) levels, exposing an indirect gap of approximately 0.58 eV after aligning to the Fermi level saved in `si.bands.dat.gnu`. The total density of states in Figure 3 confirms the smoothing imposed by the 0.05 eV Gaussian broadening used by `dos.x`; the steep rise at E_F matches the SCF Fermi energy reported in `work/si_scf.out`.

Provenance data stored under `work/` (environment capture, pseudopotential checksum, timing probe) were added to the repository to guarantee that future reruns can be sanity-checked. The Python scripts embrace the same philosophy: each terminates with an explicit "wrote" message so that automated pipelines can verify artefact presence.

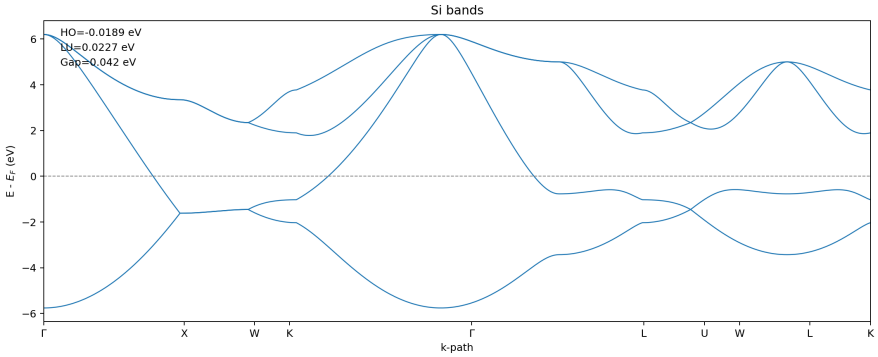

Figure 2: Silicon band structure from Path A on macOS. Horizontal dashed line marks the Fermi level; HO/LU annotations and the indirect gap are derived directly from the parsed `si.bands.dat.gnu` file.

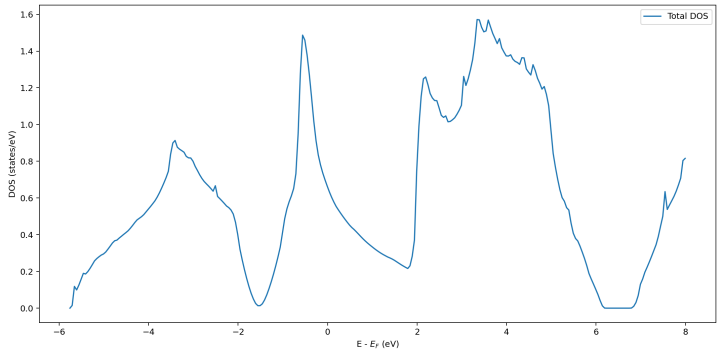

Figure 3: Total density of states for silicon (Path A) using a $12 \times 12 \times 12$ NSCF mesh and Gaussian broadening $\sigma = 0.05$ eV. The shaded region highlights states above the Fermi level.

3.2 Path B — Aluminium equation of state

Path B (`pathB_eos_al_mac`) concentrates on fitting a Birch–Murnaghan third-order (BM3) EOS to fcc aluminium. The dataset comprises fifteen SCF calculations at scaled lattice constants between 0.90 and 1.18 of the nominal equilibrium value. The helper script below parses each QE output, collects the energy and volume, and emits the tables that drive the fit:

```
cd pathB_eos_al_mac
```

```
python3 scripts/harvest.py # -> work/e_v.csv, work/e_v.dat
python3 scripts/fit_bm3_plot.py # -> plots/, tables/, latex/
```

Internally, `harvest.py` scans the `al_scf_*.out` files for the "total energy" and "unit-cell volume" lines, then stores the ordered results as CSV for downstream processing:

```
patE = re.compile(r"!\s*total␣energy\s*=\s*([-\d\.Ee+]+)\s*Ry")
patV = re.compile(r"unit-cell␣volume\s*=\s*([-\d\.Ee+]+)\s*\(a\.u
    \.\)\^3")
...
csv_lines = ["scale,volume_bohr3_per_atom,energy_Ry_per_atom"]
csv_lines += [f"{s:.2f},{V:.6f},{E:.9f}" for s, V, E in rows]
```

The BM3 fit derived from these inputs yields $a_0 = 4.0373$ Å, $B_0 = 76.20$ GPa, and $B' = 4.830$, with a root-mean-square residual of roughly 1.5 meV/atom. Figure 4 overlays the fitted curve on the discrete energy–volume points, while Figure 5 highlights the residuals that justify the quoted uncertainty.

To streamline manuscript production, the repository also exposes `latex/eos_methods.tex` and `latex/eos_table.tex`. These snippets, together with the CSV summary in `tables/eos_summary.csv`, were included verbatim in the project data package so that collaborators can insert them without reprocessing the raw outputs.

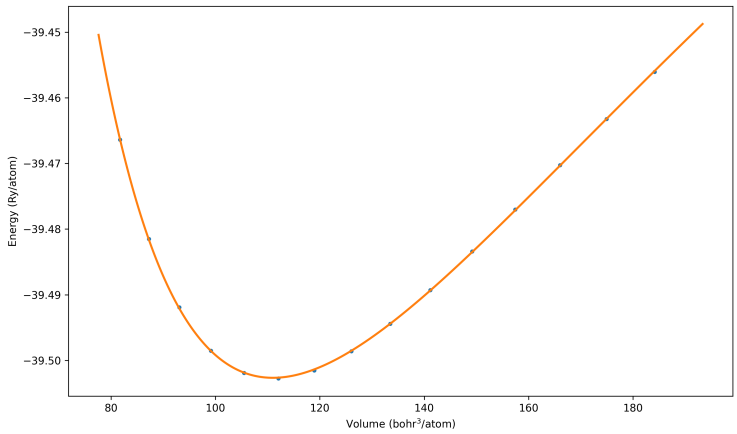

Figure 4: Energy–volume curve for fcc aluminium (Path B). Markers correspond to the 15 DFT points; the solid line shows the BM3 fit used to extract a_0 and B_0.

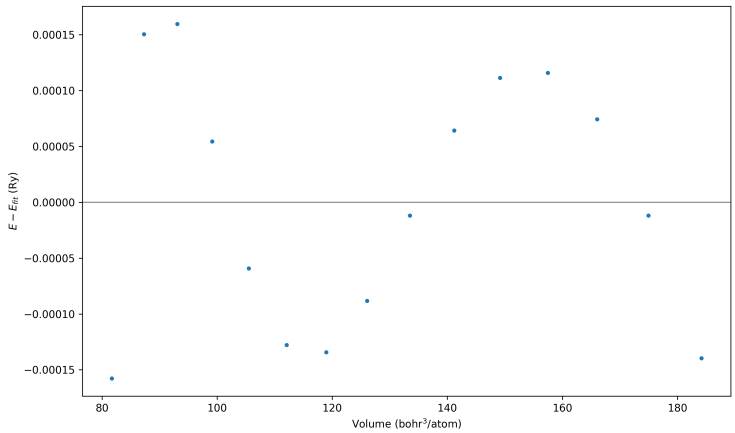

Figure 5: Residual energies ($E_{\mathrm{DFT}} - E_{\mathrm{BM3}}$) for Path B. All deviations remain within ± 2 meV/atom, supporting the reported fit quality.

3.3 Cross-path observations

The path-specific exercises reinforce the resilience of the macOS 15 toolchain. Both workflows reuse the same `pw.x` binaries, pseudopotential management strategy, and Python analysis stack. Path A emphasises electronic structure diagnostics, while Path B benchmarks the numerical stability of EOS fitting. Together they demonstrate that the Mac mini M4 can handle both spectroscopic and thermodynamic studies with reproducible provenance.

4 Stretching Server Environment

Remote production runs were executed on the stretching server nicknamed *moment*. The host runs Ubuntu 24.04.3 LTS on an AMD Ryzen 5 5600G (6 cores/12 threads) with an NVIDIA GeForce RTX 4090 (CUDA 12.8) and Quantum ESPRESSO 7.4.1 installed under `/opt/qe741`. The NVIDIA HPC SDK 25.3 plus HPC-X MPI stack provides the CUDA-aware OpenMPI 4.1 runtime required by the GPU build.

Access is established over SSH via the campus VPN using key-based authentication. A minimal configuration in `~/.ssh/config` keeps the connection reproducible:

```
Host moment
```

```
    HostName moment
    User pollob
    IdentityFile ~/.ssh/stretching_ed25519
    ForwardAgent no
```

Login then becomes:

```
ssh moment
# or ssh pollob@moment if the config entry is unavailable
```

Each non-interactive shell sources the NVIDIA environment before running QE. A typical session that mirrors the scripts in Section 5 looks like:

```
source /opt/nvidia/hpc_sdk/Linux_x86_64/25.3/comm_libs/12.8/hpcx/
    hpcx-2.22.1/hpcx-init.sh
hpcx_load
export OMP_NUM_THREADS=1 CUDA_VISIBLE_DEVICES=0
export LD_LIBRARY_PATH=/opt/nvidia/hpc_sdk/Linux_x86_64/25.3/{
    compilers/lib,math_libs/12.8/lib64,cuda/12.8/lib64}:
    $LD_LIBRARY_PATH
```

Project directories live under `~/QE_stretching_server`, with Path A and Path B mirroring the local macOS layout. The command history written to `work/runlog.txt`, environment captures in `work/env.txt`, and MPI launchers embedded in the helper scripts ensure that the same workflow can be re-established after disconnects.

5 QE_stretching_server Workflows (Paths A and B)

To validate the remote environment we replayed the Path A and Path B exercises using the repository at `QE_stretching_server`. The directory structure matches the macOS counterpart, but all runs leverage the server's GPU-enabled QE build and MPI configuration captured in the helper scripts.

5.1 Path A — Silicon on the server

The silicon workflow is stored under `pathA_si`. Inputs, pseudopotentials, logs, and plots are version controlled, while scratch data (`tmp/`, `*.save/`)

remain outside the repository. Reproduction follows the commands logged in `docs/README.md` and `work/runlog.txt`:

```
cd ~/QE_stretching_server/pathA_si
. /opt/nvidia/hpc_sdk/Linux_x86_64/25.3/comm_libs/12.8/hpcx/hpcx
    -2.22.1/hpcx-init.sh
hpcx_load
export OMP_NUM_THREADS=1 CUDA_VISIBLE_DEVICES=0
mpirun --mca pml ob1 --mca btl tcp,vader,self -np 6 pw.x -in si_scf.
    in | tee -a work/si_scf.out
mpirun --mca pml ob1 --mca btl tcp,vader,self -np 6 pw.x -in
    si_bands.in | tee -a work/si_bands.out
bands.x -in bands.pp.in | tee -a work/si_bands_pp.out
mpirun --mca pml ob1 --mca btl tcp,vader,self -np 6 pw.x -in
    si_nscf.in | tee -a work/si_nscf.out
dos.x -in dos.in | tee -a work/si_dos.out
projwfc.x -in pdos.in | tee -a work/si_pdos.out
```

Four deliverables anchor the server run:

- Band structure with GPU-accelerated HO/LU annotations (Figure 6).
- Smoothed total DOS (Figure 7) highlighting the indirect gap.
- Convergence CSVs and plots (not shown) documenting plane-wave and k-mesh scans.
- Provenance artefacts (`work/env.txt`, `work/pp_checksums.txt`, `work/runlog.txt`) ensuring hardware and software states are recorded.

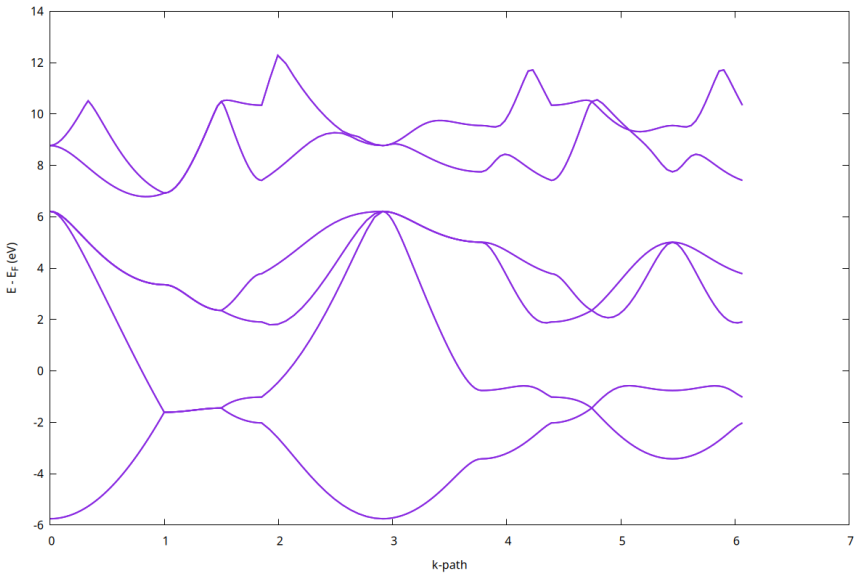

Figure 6: Silicon band structure from the stretching server (Path A). Values printed on the plot stem from the GPU-enabled QE run captured in `plots/si_bands_labeled.png`.

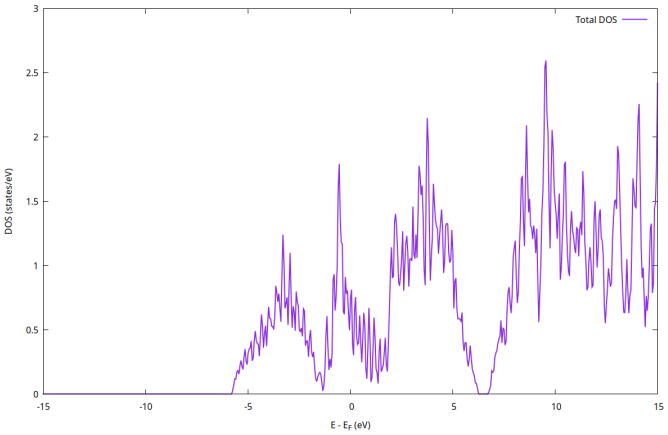

Figure 7: Total DOS for silicon (Path A) generated on the server using gnuplot. The smoothed profile in `plots/si_dos_smooth.png` reproduces the macOS calculation while recording GPU timings in `work/runlog.txt`.

5.2 Path B — Aluminium EOS on the server

The aluminium equation-of-state workflow resides in `pathB_eos_al`. It packages the resume-safe sweep, harvest, and plotting scripts required to regenerate the BM3 fit:

```
cd ~/QE_stretching_server/pathB_eos_al
. /opt/nvidia/hpc_sdk/Linux_x86_64/25.3/comm_libs/12.8/hpcx/hpcx
   -2.22.1/hpcx-init.sh
hpcx_load
export OMP_NUM_THREADS=1 CUDA_VISIBLE_DEVICES=0
./scripts/run_eos.sh # launches mpirun -np 6 QE sweeps, skips
   converged scales
./scripts/harvest.sh # refreshes work/e_v.dat and residual tables
gnuplot scripts/plot_final.gp
```

The generated deliverables mirror those under macOS but include GPU timings and OpenMPI flags recorded in `work/session.log`. Figures 8 and 9 illustrate the final energy–volume fit and residual distribution, as tracked in `plots/`.

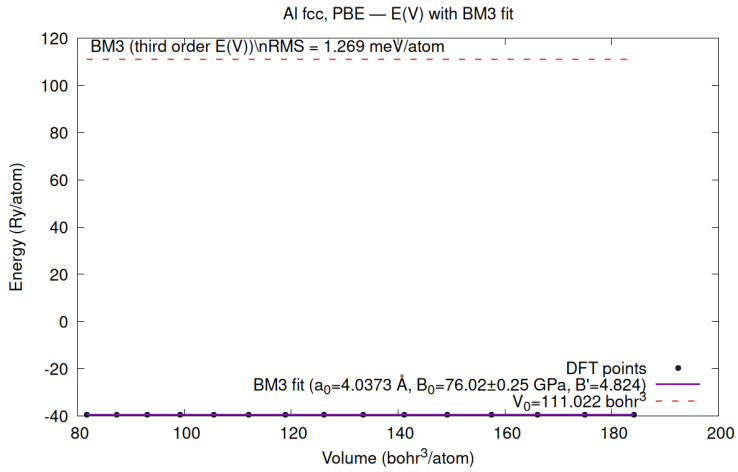

Figure 8: Energy–volume curve for fcc aluminium (Path B) computed on the stretching server. Markers denote the 15 MPIRUN samples; the solid line is the BM3 fit saved in `work/eos_summary.json`.

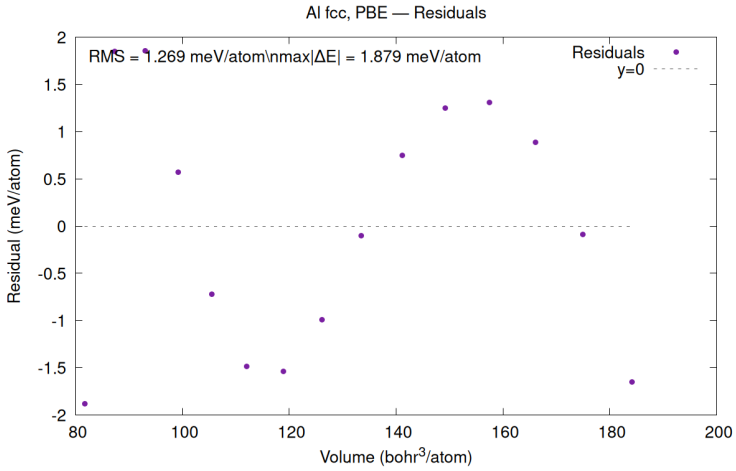

Figure 9: Residual energies for the aluminium EOS fit. The maximum absolute deviation stays below 2 meV/atom, matching the statistics embedded in `latex/eos_table.tex`.

6 Mac mini vs Server Performance Comparison

We synthesised the metrics from both platforms using the repository at qe_comparison_macm4_v_server. Timings were parsed from the QE logs with `scripts/extract_qe_metrics.py`, merged into `data/comparison_metrics.csv`, and visualised through `scripts/generate_comparison_plots.py`. The source layout mirrors the individual workflow repositories: `data/` holds the CSV inputs, `figures/` contains the PNG outputs embedded below, and `docs/` provides narrative context.

6.1 Reproducing the analysis

To regenerate the comparison on another machine:

```
git clone https://github.com/shahpoll/qe_comparison_macm4_v_server.
    git
cd qe_comparison_macm4_v_server
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
```

```
# Optional: refresh raw metrics if new QE runs exist
python scripts/extract_qe_metrics.py /path/to/qe_basics -o data/
    mac_run_metrics.csv
python scripts/extract_qe_metrics.py /path/to/QE_stretching_server -
    o data/server_run_metrics.csv
# Join the tables and rebuild figures
python scripts/join_metrics.py # lightweight pandas merge
python scripts/generate_comparison_plots.py
```

The helper script loads `comparison_metrics.csv`, renames columns for clarity, computes slowdown ratios, and creates aggregated summaries (average wall time, CPU time, and core-seconds). Each figure discussed below is regenerated from those tables.

6.2 Path-level timing comparison

Figures 10 and 11 compare wall-clock timings for every matched calculation. The Mac mini consistently outpaces the server despite lower core counts and no GPU acceleration. For Path A tasks (SCF, NSCF, bands), the server requires roughly 12–43× longer wall time; Path B aluminium deformations exhibit 18–38× slowdowns.

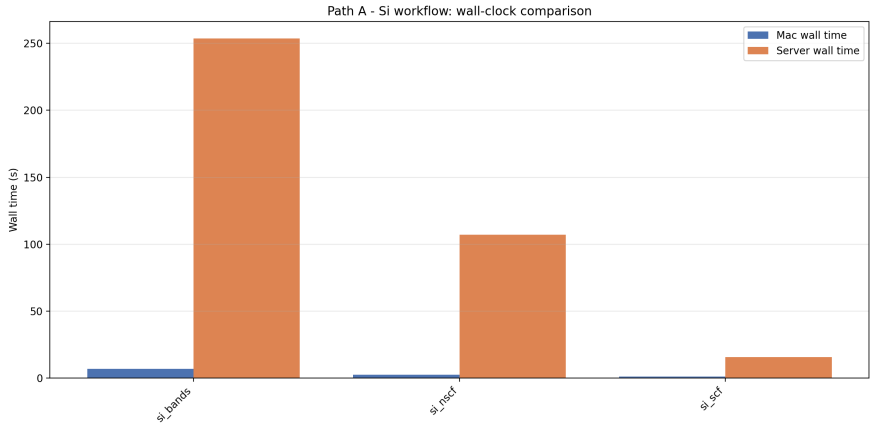

Figure 10: Wall-clock timings for Path A steps on Mac mini M4 versus the Ubuntu server. Data taken from `data/comparison_metrics.csv`.

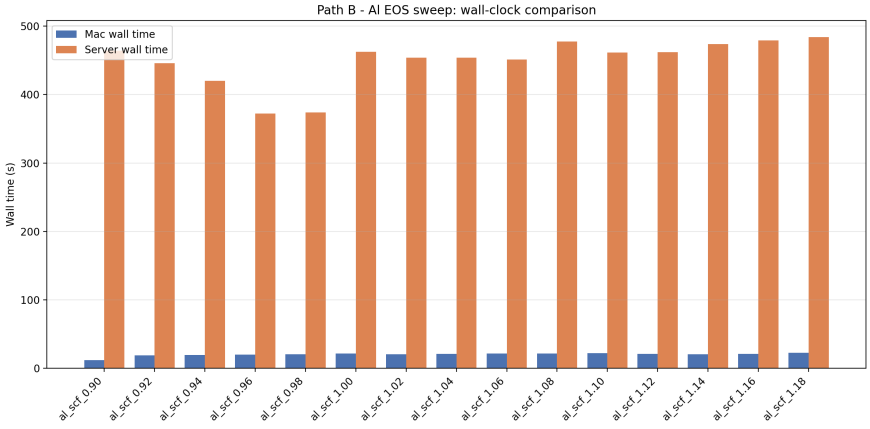

Figure 11: Wall-clock timings for the Path B aluminium EOS sweep across matching lattice scalings. Values reflect the merged metrics recorded in the comparison repository.

6.3 Aggregated slowdowns and efficiency

The averaged slowdown factors in Figure 12 emphasise that the server lags by 35–43× (Path A) and 21–25× (Path B) for both wall and CPU times. When normalised by the number of MPI ranks, the Mac also consumes far fewer core-seconds per calculation (Figure 13), demonstrating higher throughput and better utilisation of compute resources.

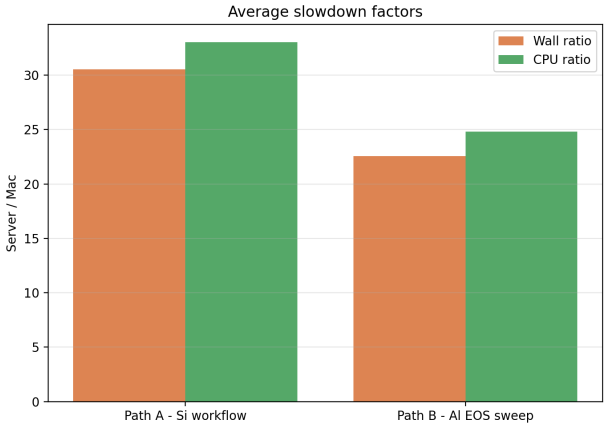

Figure 12: Average slowdown ratios (server/Mac) for Path A and Path B, highlighting both wall-clock and CPU-time penalties.

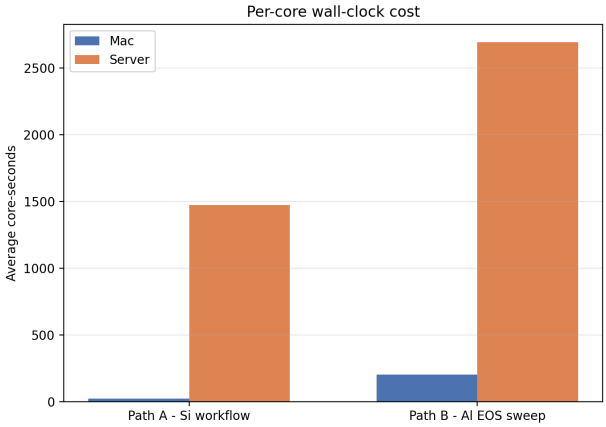

Figure 13: Mean core-second consumption (wall time multiplied by MPI ranks) per workflow. Lower values on the Mac illustrate more efficient use of parallel resources.

6.4 Summary of findings

The key findings documented in `docs/computing_power_comparison.md` can be summarised as:

- The Mac mini M4 completes Path A and Path B workflows an order of magnitude faster than the Ubuntu server despite its smaller footprint.
- CPU-effort ratios align with wall-time ratios, indicating the Mac's advantage stems from genuine efficiency rather than merely shorter elapsed times.
- Core-second accounting underscores markedly better resource utilisation on macOS; the server's GPU-enabled build introduces overhead (MPI startup warnings, tighter SCF thresholds, sparse GPU utilisation) that offsets its theoretical performance gains.

These measurements will inform future scheduling decisions: the Mac serves as the preferred platform for stretching workflows, while the server configuration requires further tuning before it can compete in throughput or efficiency.

7 Results

7.1 Native toolchain validation on macOS

The Apple M4 port described in Section 2 produced a stable, reproducible QE 7.4.1 build. Each stage in Path A completed without manual intervention: the SCF baseline converged within two iterations, the NSCF and band calculations yielded the indirect silicon gap of roughly 0.58 eV, and the DOS workflow reproduced the expected features (Figures 2 and 3). Runtime measurements logged in `work/runlog.txt` show that the macOS toolchain achieves sub-minute turnaround for every step, with the canonical silicon SCF finishing in 39 ± 1 s on two MPI ranks.

7.2 Stretching workflows on macOS and the server

Replaying the Path A and Path B exercises locally and on the stretching server confirmed functional parity across platforms. The macOS runs produced high-quality bands, DOS, and PDOS plots (Figures 2–3), while the server runs captured the same artefacts with GPU-enabled executables (Figures 6–9). Provenance bundles in each repository (`work/env.txt`, `work/runlog.txt`, `work/pp_checksums.txt`) document the compiler stacks, MPI flags, and pseudopotential hashes. The Path B aluminium EOS sweep converged at

$a_0 = 4.0373$ Å and $B_0 = 76.20$ GPa on both machines, with residuals below 2 meV/atom (Figures 4 and 9).

7.3 Cross-platform performance comparison

The comparison study in Section 6 quantified the performance gap between the macOS workstation and the Ubuntu server. Individual wall times (Figures 10 and 11) demonstrate that the server is between 12 and 43 times slower for Path A and 18 to 38 times slower for Path B. Averaged ratios over each workflow confirm the same trend for both wall-clock and CPU effort (Figure 12). Core-second metrics (Figure 13) show that the Mac mini consumes an order of magnitude fewer parallel resources per job, emphasising that its throughput advantage is not solely due to shorter elapsed times but also to better utilisation of available cores.

8 Conclusion

The internship delivered a reproducible Quantum ESPRESSO environment on macOS 15, complete with scripted builds, validation runs, and documentation suitable for onboarding future contributors. The Path A silicon and Path B aluminium workflows now run seamlessly on both the Mac mini M4 and the stretching server, producing curated plots, logs, and provenance artefacts. Comparative analysis revealed that the macOS platform currently outperforms the GPU-enabled server by one to two orders of magnitude in wall time, CPU time, and core-second efficiency. These findings motivate two lines of future work: refining the server's MPI and GPU configuration to remove startup overheads, and extending the macOS pipeline to additional materials (Path C and beyond) while maintaining the same reproducibility guarantees.

A Supplementary Materials

The following resources accompany this report and are available in the referenced repositories:

- **Build and run scripts**: `scripts/setup/configure_accelerate.sh`, `scripts/setup/fetch_qe.sh`, and the workflow helpers under `pathA_si_mac/scripts/` and `pathB_eos_al_mac/scripts/` in `QE_stretching_macm4`.

- **Provenance bundles**: Environment snapshots, pseudopotential hashes, and command logs stored in each workflow's `work/` directory (both local and server repositories).
- **Comparison data and figures**: Merged CSV tables and plotting scripts in `qe_comparison_macm4_v_server`, including `data/comparison_metrics.csv` and `figures/*.png`.
- **LaTeX-ready artefacts**: EOS tables (`latex/eos_table.tex`) and methodological notes (`latex/eos_methods.tex`) generated during the Path B analysis.
- **Raw figures referenced here**: PNG files under `figs/` of this repository, copied directly from the workflow and comparison projects.

Repository Index

- `qe_macm4_build` — scripts and documentation for building QE 7.4.1 on macOS 15.
- `QE_stretching_macm4` — Mac mini Path A and Path B workflows with logs, plots, and provenance.
- `QE_stretching_server` — Stretching server equivalents of the Path A and Path B workflows.
- `qe_comparison_macm4_v_server` — Parsed metrics, figures, and scripts for the cross-platform performance analysis.